Papa

Papa

The Personal AI Project Assistant is a lightweight, single-user task and follow-up system designed to help manage commitments, delegation, and ongoing communication across many concurrent projects, meetings, and conversations.

The product focuses on **accountability, follow-up, and daily execution**, rather than traditional project management.

PRD

Product Requirements Document (PRD)

Product: Personal AI Project Assistant

Version: 1.1

Owner: Brett Burbidge

Status: Draft – Ready for Review

---

1. Overview

1.1 Purpose

The Personal AI Project Assistant is a lightweight, single-user task and follow-up system designed to help manage commitments, delegation, and ongoing communication across many concurrent projects, meetings, and conversations.

The product focuses on **accountability, follow-up, and daily execution**, rather than traditional project management.

1.2 Problem Statement

The user regularly receives action items during meetings, phone calls, and informal conversations. These tasks often:

- Lack clear deadlines
- Require follow-up with multiple people
- Are delegated but still require oversight
- Are forgotten due to cognitive overload

Existing tools fail to model:

- Accountability separate from execution
- Stakeholders and follow-ups as first-class concepts
- Dynamic urgency as deadlines approach
- Assisted communication when timelines are unclear

1.3 Goals

- Prevent tasks and commitments from being forgotten
- Ensure delegation includes structured follow-up
- Provide clear daily and weekly focus
- Reduce mental load through automation
- Enable confident communication with stakeholders

1.4 Non-Goals (v1)

- Multi-user collaboration
- Time tracking or effort estimation
- Full project management workflows
- CRM or ticketing system functionality
- Autonomous message sending

---

2. Core Concepts & Definitions

2.1 Task

A **Task** is a unit of work that needs to be completed.

Required Attributes

- Title
- Accountable
- Status

Optional Attributes

- Description / context
- Due date
- Project
- Executor
- Priority
- Stakeholders
- Follow-up rules

2.2 Roles

2.2.1 Accountable

The **Accountable** person is responsible for ensuring the task is completed and for communicating status.

Rules:

- Every task has exactly one Accountable
- The Accountable may or may not be the Executor

- The Accountable is the default follow-up owner

Default:

- The user is Accountable unless changed

2.2.2 Executor

The **Executor** is the person performing the work.

Rules:

- Optional
- One per task (v1)
- May be the same as the Accountable
- Used to generate progress follow-ups

2.2.3 Stakeholder

A **Stakeholder** is any person who expects updates or results related to the task.

Rules:

- Zero or more stakeholders per task
- Stakeholders may include:
    - The original requester
    - A manager
    - A collaborator
    - The user themselves

- Stakeholders apply to both self-executed and delegated tasks

---

2.3 Follow-Up

A **Follow-Up** is a reminder or communication action tied to a task.

Characteristics:

- Always linked to a task
- May target the Accountable, Executor, or Stakeholders
- Generated automatically or manually
- Not a standalone task

2.4 Project

A **Project** is a loose grouping of tasks.

Rules:

- Tasks may exist without a project
- Tasks can move freely between projects
- Projects impose no workflow constraints
- Projects may optionally have due dates (informational only)

---

3. Task Lifecycle

3.1 Task Creation

Tasks can be created via:

- Manual entry
- AI-assisted capture
- Free-text input

Defaults:

- Accountable = User
- Status = Not Started
- Executor = User (unless delegated)

3.2 Execution Decision

Each task requires a decision:

Who is executing this task?

Self-Executed Task

- Accountable = User
- Executor = User
- Stakeholders may be present

System generates:

- Self accountability reminders
- Stakeholder update follow-ups

Delegated Task

- Accountable = User

- Executor = Another person
- Stakeholders optional

System generates:

- Executor progress follow-ups
- Stakeholder update follow-ups
- Accountability reminders

3.3 Task Statuses

Supported statuses:

- Not Started
- In Progress
- Waiting
- Completed
- Cancelled

Status affects follow-up generation.

---

4. Follow-Up Engine

4.1 Follow-Up Categories

Each task may generate one or more of the following:

1. **Self Accountability Follow-Ups**
    - Confirm completion or reassess scope

2. **Executor Follow-Ups**
    - Check progress when Executor ≠ Accountable
3. **Stakeholder Follow-Ups**
    - Provide updates or confirm expectations

4.2 Follow-Up Generation Rules

Follow-ups are generated based on:

- Due date
- Time remaining
- Task status
- Presence of Executor
- Presence of Stakeholders

Default cadence:

- Due in >14 days → weekly
- Due in 7–14 days → every 3 days
- Due in <7 days → daily
- Overdue → daily escalation

Rules are:

- Editable per task
- Overrideable by the user

---

5. Daily & Weekly Views

5.1 Daily Board

The Daily Board is the primary working interface.

Features:

- Tasks ordered by urgency
- Follow-ups displayed inline
- Overdue items pinned to top
- Incomplete tasks roll forward automatically

Actions:

- Drag to reprioritize
- Mark complete
- Snooze
- Adjust follow-up rules

5.2 Weekly View

A calendar-style view showing:

- Tasks by due date
- Visual urgency indicators
- Delegated vs self-executed tasks

---

6. AI Assistant Capabilities

6.1 Context Management

Each task and project supports stored context:

- Notes
- Meeting summaries
- Links
- People involved

Context is used for AI reasoning and message drafting.

6.2 AI Actions

The AI assistant can:

- Parse free-text input into tasks and projects
- Suggest Accountable, Executor, and Stakeholders
- Propose due dates when unclear
- Automatically suggest follow-up rules based on timing and context
- Draft stakeholder updates and executor check-ins
- Ask clarifying questions when intent is ambiguous

All AI suggestions require user confirmation and are editable.

---

7. People & Mentions

- People are first-class entities
- @mentions supported in tasks and AI prompts

- Roles assigned per task
- Follow-ups can target specific people

---

8. Notifications & Reminders

- Follow-ups surface in the Daily Board
- No autonomous messaging in v1
- Escalation behavior for overdue tasks

---

9. Success Criteria

Qualitative:

- Reduced missed follow-ups
- Increased clarity of daily priorities
- Improved confidence in communication

Quantitative (future):

- On-time task completion rate
- Follow-up frequency per task
- Reduction in overdue tasks

---

10. Decisions & Constraints

- One Executor per task (v1)
- Project due dates are informational only
- Automation-first with manual override
- AI is assistive, not authoritative

Core Data Model

Core Data Model (v1)

Below is a pragmatic relational model that supports: Tasks, Projects, People, Stakeholders, Follow-Ups, and AI context. It keeps v1 simple (single user, 1 executor, many stakeholders) and leaves room for later multi-user expansion.

---

1) Entities

1.1 Person

Represents anyone you can @mention (including you).

Person

- id (uuid, PK)
- handle (text, unique) — e.g., “brett”, “steen”
- display_name (text)
- email (text, nullable)
- source (text) — e.g., “manual”, “import”
- created_at (timestamptz)
- updated_at (timestamptz)

Notes:

- Keep handle unique for mention resolution.

1.2 Project

Loose grouping for tasks.

Project

- id (uuid, PK)
- name (text, required)
- description (text, nullable)
- due_at (timestamptz, nullable) — informational only (v1)
- is_active (bool, default true)
- created_at (timestamptz)
- updated_at (timestamptz)

1.3 Task

The central object.

Task

- id (uuid, PK)
- title (text, required)
- description (text, nullable)
- status (enum: not_started, in_progress, waiting, completed, cancelled)
- priority (int, nullable) — optional/manual
- due_at (timestamptz, nullable)
- project_id (uuid, FK -> Project.id, nullable)

Roles:

- accountable_person_id (uuid, FK -> Person.id, required)

- executor_person_id (uuid, FK -> Person.id, nullable) — v1: single executor

Scheduling / board:

- board_date (date, nullable) — which day it appears on the Daily Board
- sort_order (numeric, nullable) — drag/drop ordering within board_date

Lifecycle:

- completed_at (timestamptz, nullable)
- created_at (timestamptz)
- updated_at (timestamptz)

Constraints:

- accountable_person_id required
- executor_person_id optional, but if null implies self-executed unless otherwise inferred

---

1.4 TaskStakeholder (join table)

Supports 0..N stakeholders per task (including possibly yourself).

TaskStakeholder

- task_id (uuid, FK -> Task.id)
- person_id (uuid, FK -> Person.id)
- role (enum, nullable) — e.g., requester, informed, approver (optional in v1)
- created_at (timestamptz)

PK / Unique:

- Composite PK (task_id, person_id)

1.5 FollowUpRule

Defines the cadence logic for a task. Keep it task-scoped for v1.

FollowUpRule

- id (uuid, PK)
- task_id (uuid, FK -> Task.id, unique) — v1 assumes one active rule set per task
- mode (enum: default, custom) — how it was chosen
- generated_by (enum: ai, user)
- config_json (jsonb) — stores rule parameters (see below)
- created_at (timestamptz)
- updated_at (timestamptz)

Example config_json:

```
{
  "cadence": [
    { "when_days_to_due_gt": 14, "every_days": 7 },
    { "when_days_to_due_between": [7, 14], "every_days": 3 },
    { "when_days_to_due_lt": 7, "every_days": 1 },
    { "when_overdue": true, "every_days": 1 }
  ],
  "include_executor_followups": true,
  "include_stakeholder_followups": true
}
```

1.6 FollowUp (instances)

An actual follow-up item that appears on the board / prompts a message draft.

FollowUp

- id (uuid, PK)
- task_id (uuid, FK -> Task.id, required)
- type (enum: self_accountability, executor_checkin, stakeholder_update)
- target_person_id (uuid, FK -> Person.id, nullable)
- due_at (timestamptz, required) — when the follow-up should surface
- status (enum: pending, done, snoozed, cancelled)
- snoozed_until (timestamptz, nullable)
- last_generated_at (timestamptz, nullable) — if regenerated by engine
- created_at (timestamptz)
- updated_at (timestamptz)

Rules:

- Stakeholder follow-ups create one FollowUp per stakeholder (or allow batching later)
- Executor follow-ups target executor_person_id
- Self follow-ups target accountable_person_id (often you)

1.7 ContextItem

Stores information used by the AI agent (notes, meeting excerpts, links).

ContextItem

- id (uuid, PK)
- scope (enum: task, project)
- task_id (uuid, FK -> Task.id, nullable)

- project_id (uuid, FK -> Project.id, nullable)
- kind (enum: note, meeting_summary, link, email_draft, message_draft, attachment_ref)
- content (text) — or jsonb if you prefer
- source (text, nullable) — “manual”, “import”, “ai”
- created_at (timestamptz)

Constraints:

- If scope=task then task_id required and project_id null
- If scope=project then project_id required and task_id null

---

2) Key Behaviors Supported by This Model

2.1 “Who asked me to do this?”

- Store the requester as a **stakeholder** (TaskStakeholder.role = requester).
- Task remains self-executed with Executor = you.

2.2 Delegation path

- accountable_person_id = you
- executor_person_id = someone else
- FollowUp engine generates:
    - executor_checkin follow-ups
    - stakeholder_update follow-ups (if stakeholders exist)

    - self_accountability follow-ups

2.3 Daily Board rollover

- Task.board_date is “today” if active and not completed.
- On end-of-day rollover, incomplete tasks with board_date=today become board_date=tomorrow (or next working day).
- FollowUps are driven by FollowUp.due_at and show when due.

2.4 Drag/drop ordering

- Task.sort_order (numeric) per board_date
- When you drag, update sort_order

---

3) Enums (Recommended)

- task_status: not_started, in_progress, waiting, completed, cancelled
- followup_type: self_accountability, executor_checkin, stakeholder_update
- followup_status: pending, done, snoozed, cancelled
- context_scope: task, project
- context_kind: note, meeting_summary, link, email_draft, message_draft, attachment_ref
- rule_mode: default, custom
- rule_generated_by: ai, user

---

4) Minimal Constraints / Indexes

Constraints

- Task.accountable_person_id NOT NULL
- Composite unique on TaskStakeholder(task_id, person_id)
- FollowUpRule.task_id UNIQUE (v1: one active ruleset per task)

Indexes

- Task(board_date, sort_order)
- Task(due_at)
- FollowUp(due_at, status)
- FollowUp(task_id)
- TaskStakeholder(person_id)
- Person(handle) unique

---

5) Optional v1.1 Additions (Only if needed soon)

- Task.created_from (enum: meeting, manual, ai)
- Task.requested_at (timestamptz) — if you want “asked on” explicitly
- FollowUp.template_kind (enum: email, teams) for drafting

Tech Stack

Tech Stack

1) Tech stack

App

- SvelteKit + TypeScript

DB

- Postgres (local via Docker Compose)

ORM

- Prisma

UI

- Tailwind CSS
- shadcn-svelte (components)
- Drag/drop: start simple (up/down + “move to date”), add a Svelte DnD library after flows are stable

Validation + dates

- Zod
- date-fns

AI

- OpenAI SDK (server-side only)

Tooling

- ESLint + Prettier

---

2) Repo layout (single monolith)

```
assistant-app/
  docker/
    docker-compose.yml
  prisma/
    schema.prisma
  src/
    lib/
      db/                # prisma client
      domain/            # types, enums, rules
      services/          # follow-up engine + AI service
      ui/                # reusable components
    routes/
      api/
        tasks/...
        projects/...
        people/...
        followups/...
        ai/...
      (app pages)
        board/
        week/
        inbox/
        projects/
  .env
  package.json
```

3) Environment variables (.env)

Local only:

- DATABASE_URL=postgresql://app:app@localhost:5432/ai_assistant?schema=public
- OPENAI_API_KEY=... (optional until you wire AI)
- OPENAI_MODEL=gpt-4.1-mini (or whatever you pick later)

Notes:

- Keep all OpenAI calls in **server routes** or src/lib/services/ai.ts.

---

4) Docker Compose (local Postgres)

Use a single Postgres container + volume.

- Postgres 16 (recommended)
- Expose 5432
- Volume for persistence
- Optional: admin tool (pgAdmin) if you want, but DBeaver is fine

---

5) SvelteKit data flow (minimal)

Use SvelteKit **server routes** as your API:

- +page.server.ts for page actions (create/update)
- src/routes/api/**/+server.ts for fetch-based endpoints (drag/drop reorder, follow-up generation, AI suggestions)

No separate backend service.

---

6) Core modules to implement (aligned to your model)

6.1 Prisma DB client

src/lib/db/prisma.ts

- Singleton PrismaClient

- Safe for dev hot reload

6.2 Domain types/enums

src/lib/domain/*

- task status enums
- follow-up types/status
- rule config shape (Zod schema for FollowUpRule.config_json)

6.3 Follow-up engine service

src/lib/services/followupEngine.ts

Responsibilities:

- Given a task (due date, status, accountable/executor, stakeholders), compute next follow-ups
- Create/update FollowUp rows idempotently
- Rollover logic (board_date) handled separately or in same service

6.4 AI service

src/lib/services/ai.ts

Responsibilities:

- Parse free-text task input into suggested fields (title, due date, stakeholders)
- Suggest follow-up rule config (cadence) using current date + due date
- Return a structured JSON response validated by Zod

---

7) Pages (v1 UI surfaces)

1. **Inbox**: quick add tasks (free text + optional context)
2. **Daily Board**: today's tasks + due follow-ups + reorder + complete + snooze
3. **Week View**: tasks by due date (calendar-ish list)
4. **Projects**: list projects + project detail with tasks

---

8) "Dependencies list" (npm packages)

Baseline:

- @prisma/client, prisma
- zod
- date-fns
- tailwindcss, postcss, autoprefixer
- openai (later when ready)
- eslint, prettier

UI:

- shadcn-svelte (plus its required peer deps)
- Drag/drop: postpone until after you like the UX; start with reorder controls

Build Plan

Build Plan

Scope lock (v1)

- Controls-first ordering (no drag/drop yet)
- No auth
- Single local user
- AI suggestions are assistive + confirmable (no autonomous sending)

---

1) Milestones (build order)

Milestone 0 — Repo + local DB (day 0)

Deliverable: app boots, DB reachable, Prisma working.

1. Create SvelteKit (TS) project
2. Add Tailwind
3. Add shadcn-svelte
4. Add Docker compose Postgres
5. Add Prisma + initial schema + migrate
6. Seed your own Person record (e.g., @brett)

Milestone 1 — Core CRUD + Inbox (day 1)

Deliverable: create tasks quickly, view them, edit basics.

Pages

- /inbox
    - Free-text “quick add” (title + optional details)
    - Optional fields: due date, project, stakeholders, executor
    - “Create Task” action

Minimal operations

- Create Person
- Create Project
- Create Task
- Add/remove stakeholders on a task

Milestone 2 — Daily Board + rollover (day 2)

Deliverable: today’s work surface that doesn’t lose tasks.

Page

- /board
    - Shows tasks where board_date = today and status in (not_started, in_progress, waiting)
    - Shows follow-ups due today (from FollowUp table)
    - Controls:
        - Move up/down (sort_order)
        - Mark complete

        - Snooze follow-up (e.g., +1 day / +3 days)
        - Move task to another date (tomorrow / pick date)

Background behavior (manual trigger for v1)

- "Rollover" button:
    - Any incomplete tasks on today that aren't done → set board_date = tomorrow
    - (Later you can run it automatically at midnight)

Milestone 3 — Follow-Up Engine (rules-based) (day 3)

Deliverable: system generates FollowUp instances from task + rule config.

UI

- Task detail sidebar/modal (or Task edit page)
    - Shows follow-up rule summary
    - "Regenerate follow-ups" button (v1 simple)

Engine behaviors

- For each active task:
    - Create/update upcoming follow-ups for:
        - Self accountability
        - Executor check-ins (if executor != accountable)
        - Stakeholder updates (one per stakeholder)
- Ensure idempotency: regenerating doesn't duplicate

Milestone 4 — Week View + Projects (day 4)

Deliverable: see commitments by due date and by project.

Pages

- /week
    - Lists tasks grouped by date (due_at)
    - Highlight due-soon / overdue
- /projects
    - Project list
    - /projects/[id] shows tasks under project, project context notes

---

Milestone 5 — AI Suggestions (day 5)

Deliverable: AI helps create tasks + follow-up rules, always confirmable.

UI additions

- Inbox: “AI Assist” toggle
- AI Suggestion panel:
    - Title, due date, project, executor, stakeholders
    - Follow-up cadence proposal
    - Accept / Edit / Apply

---

2) Routes to implement (SvelteKit minimal API)

2.1 Tasks

- GET /api/tasks?boardDate=YYYY-MM-DD
- GET /api/tasks?dueFrom=...&dueTo=...
- GET /api/tasks/:id
- POST /api/tasks

  Body: { title, description?, dueAt?, projectId?, accountablePersonId, executorPersonId?, stakeholderPersonIds?, boardDate? }
- PATCH /api/tasks/:id

  Update core fields
- POST /api/tasks/:id/complete

  Sets status + completed_at
- POST /api/tasks/:id/move

  Body: { boardDate, sortOrder? }
- POST /api/tasks/:id/reorder

  Body: { direction: "up" | "down" } (simple v1)

2.2 Projects

- GET /api/projects
- POST /api/projects
- GET /api/projects/:id

- PATCH /api/projects/:id

2.3 People

- GET /api/people?query=...
- POST /api/people (create by handle/name)
- GET /api/people/resolve?handle=@steen (optional convenience)

2.4 Stakeholders

- POST /api/tasks/:id/stakeholders Body: { personId }
- DELETE /api/tasks/:id/stakeholders/:personId

2.5 Follow-ups

- GET /api/followups?dueDate=YYYY-MM-DD
- POST /api/followups/:id/snooze Body: { until }
- POST /api/tasks/:id/followups/generate (regenerate for one task)
- POST /api/followups/generate?boardDate=YYYY-MM-DD (generate for all active tasks)

2.6 Rollover

- POST /api/board/rollover Body: { fromDate, toDate }

2.7 AI

- POST /api/ai/suggest-task

  Body: { inputText, currentDate, knownPeople?, knownProjects? }

Returns: { suggestedTaskFields, suggestedStakeholders, suggestedFollowUpRuleConfig }

- POST /api/ai/draft-message

  Body: { taskId, targetPersonId, channel: "email"|"teams", intent: "stakeholder_update"|"executor_checkin" }

  Returns: { subject?, body }

---

3) Service function signatures (so code stays organized)

3.1 Follow-up engine

src/lib/services/followupEngine.ts

- generateForTask(taskId: string, asOf: Date): Promise<void>
- generateForActiveTasks(boardDate: Date, asOf: Date): Promise<void>
- computeCadence(task, ruleConfig, asOf): FollowUpSpec[]

  (FollowUpSpec includes type, target_person_id, due_at)

Idempotency approach (v1):

- When generating, look ahead N days (e.g., 21)
- For each computed follow-up, upsert by (task_id, type, target_person_id, due_at::date) or similar

3.2 Rollover

src/lib/services/board.ts

- rollover(fromDate: Date, toDate: Date): Promise<number> (returns number moved)

3.3 AI service

src/lib/services/ai.ts

- suggestTask(inputText: string, asOf: Date): Promise<SuggestTaskResult>
- suggestFollowUpRule(taskDraft: TaskDraft, asOf: Date): Promise<RuleConfig>
- draftMessage(taskId: string, targetPersonId: string, intent: ..., channel: ...): Promise<DraftMessage>

All responses validated by Zod before returning to UI.

---

4) Minimum UI interactions per page

/inbox

- Textbox: “Add a task…”
- Optional: expand panel (due date, project, executor, stakeholders)
- Buttons:
    - Create
    - AI Suggest (optional)
- After create: redirect to /board

/board

- Sections:

    - Tasks (today)
    - Follow-ups (due today)
- Task row controls:
    - Up / Down
    - Complete
    - Move to tomorrow / pick date
    - Open details
- Follow-up row controls:
    - Done
    - Snooze +1 / +3 / pick date
    - “Draft message” (later when AI added)
- Top actions:
    - Rollover (moves incomplete to tomorrow)
    - Generate follow-ups (manual trigger)

/week

- Group tasks by due date (including “No due date”)
- Quick edit due date
- Jump to task details

/projects

- Create project

- Project detail shows:
    - Tasks in project
    - Project context notes

---

5) Implementation choices to keep v1 smooth

- Use SvelteKit form actions for create/edit where possible; use /api/* endpoints for:
    - reorder
    - rollover
    - follow-up generation
    - AI calls
- Keep follow-up generation manual-trigger initially (button) so you can iterate safely
- Store follow-up rule config in FollowUpRule.config_json even if “default” at first

---

6) Definition of Done (v1)

You can:

- Add a task quickly (with stakeholders and due date)
- See it on Daily Board and Week View
- Reorder tasks (up/down)

- Mark tasks complete
- Roll tasks forward to tomorrow
- Generate follow-ups and see them appear
- Snooze/complete follow-ups
- (Optional v1+) Use AI to suggest stakeholders + follow-up cadence and apply it